CPU Scheduling

Introduction and Basic Concepts

Introduction

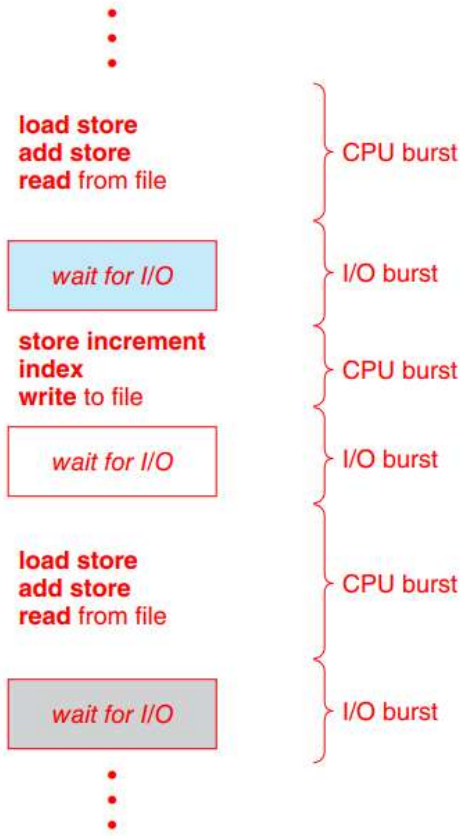
- In a multiprogrammed system, multiple processes compete for the single resource: the CPU
- The primary function of the CPU scheduler is to maximize system efficiency while providing fast, fair, and reliable service to all users
- Topic for this section covers the fundamental concepts, performance criteria, and major algorithms used by operating systems to manage CPU time.

Basic Concept

- **Single-Core Limitation:** In a system with one CPU core, only one process can run at any given moment
  - Other processes must wait for the CPU to become available.
- **Goal of Multiprogramming**
  - The primary objective of multiprogramming is to maximize CPU utilization by ensuring that a process is running at all times
- **How Multiprogramming Works**
  - When a running process has to wait (e.g., for an I/O request to complete), the operating system takes the CPU away from that process and gives it to another process that is ready to run
  - This prevents the CPU from sitting idle.
- **Memory Requirement**
  - Multiprogramming requires that multiple processes are loaded and kept in memory simultaneously.
- **Extension to Multicore**
  - The concept of keeping the CPU busy extends to all processing cores in a multicore system.
- **Scheduling's Importance**
  - The scheduling of processes is a fundamental function of an operating system, and CPU scheduling is a central aspect of operating-system design.

CPU-I/O Burst Cycle

- Process execution consists of a cycle of CPU execution (CPU burst) and I/O wait (I/O burst)
  - Processes alternate between these two states
  - **CPU-bound jobs** have long CPU bursts
  - **I/O-bound jobs** have many short CPU bursts



CPU Scheduling

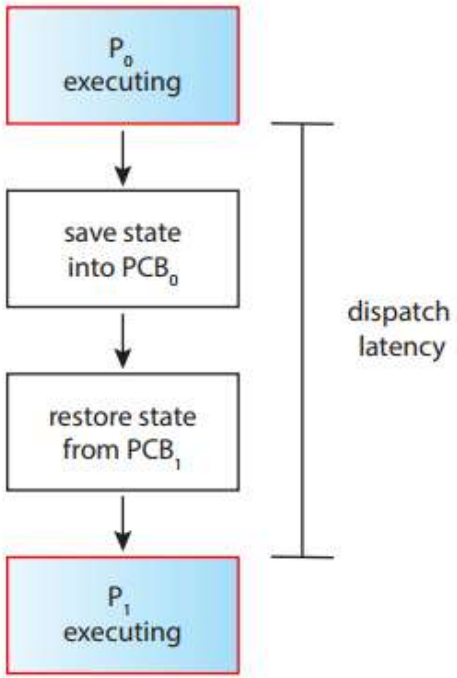
- CPU-scheduling decisions may take place under the following four circumstances:
  - When a process switches from the running state to the waiting state (for example, as the result of an I/O request or an invocation of wait() for the termination of a child process)
  - When a process switches from the running state to the ready state (for example, when an interrupt occurs)
  - When a process switches from the waiting state to the ready state (for example, at completion of I/O)
  - When a process terminates
- For first and last situations, a new process (if one exists in the ready queue) must be selected for execution
- A preemptive kernel requires mechanisms such as mutex locks to prevent race conditions when accessing shared kernel data structures

CPU Scheduler

- **CPU Scheduler (Short-Term Scheduler):** Selects a process from the **ready queue** and allocates the CPU to it
  - Ready queue is not necessarily a first-in, first-out (FIFO) queue
- **Preemptive vs. Nonpreemptive Scheduling:**
  - **Nonpreemptive:** Once the CPU is allocated to a process, it holds the CPU until it voluntarily releases it (e.g., termination or waiting for I/O)
  - **Preemptive:** The OS can interrupt a running process and force it to release the CPU (e.g., when a higher-priority task arrives)
    - This requires careful design to prevent data corruption when kernel data is shared
- Virtually all modern operating systems (Windows, macOS, Linux, and UNIX) use pre-emptive scheduling algorithms
- Pre-emptive scheduling can result in race conditions when data are shared among several processes

Dispatcher

- The kernel module responsible for giving control of the CPU to the process selected by the CPU Scheduler
- It performs the crucial act of switching processes
- Its critical function involves these three steps:
  - Context Switching (saving the state of the old process and loading the PCB of the new process).
  - Switching to user mode.
  - Jumping to the proper location in the user program to restart execution.
- **Dispatch Latency:** The time taken by the dispatcher to stop one process and start another running
  - This time is considered pure overhead as no useful work is done



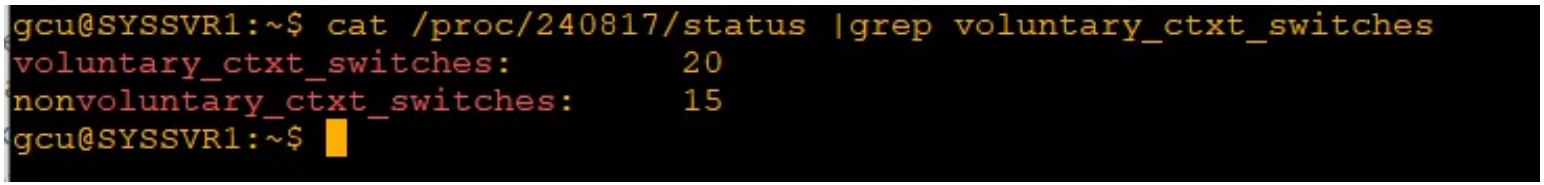
Linux `vmstat`

- Number of context switch is shown under “system” in column “cs”

```
gcu@SYSSVR1:~$ vmstat 1 3
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0      0 5383072      0 1534396    0    0    31    41    9    3  0  0 99  0  0
 1  0      0 5383408      0 1534396    0    0     0     0 1561 2934  0  0 100  0  0
 1  0      0 5383408      0 1534396    0    0     0   136 1579 3026  0  0 99  0  0
gcu@SYSSVR1:~$
```

Linux `proc` file system

- Linux proc file system using the PID can show the context switches of a process



Scheduling Criteria

- The goal of CPU scheduling is to maximize and minimize certain criteria:
- **Maximize:**
  - **CPU Utilization:** Keep the CPU as busy as possible (range from 0 to 100%).
  - **Throughput:** The number of processes completed per time unit.
- **Minimize:**
  - **Turnaround Time:** The total time from submission to completion (includes waiting in queues, execution, and I/O).
  - **Waiting Time:** The total time a process spends waiting in the ready queue.
  - **Response Time:** In an interactive system, the time from submission of a request until the first response is produced (time to start responding, not time to complete).

Scheduling Algorithm Non-Preemptive

First Come First Serve

- **Operation:** The process that requests the CPU first is allocated the CPU first. It is implemented with a FIFO queue.
- **Performance:** Simple but can result in poor performance.
- **Convoy Effect:** If a single long CPU-bound process arrives first, it can block all short processes behind it, leading to low CPU and device utilization.

First Come First Serve

Process	Burst Time
P_1	24
P_2	3
P_3	3



Average waiting time = (P1 wait time + P2 wait time + P3 wait time)/3
Average waiting time = (0 + 24 + 27) / 3
Average waiting time = 17

Assume all values are in ms

First Come First Serve (2nd Scenario)

Process	Burst Time
P_1	24
P_2	3
P_3	3



Average waiting time = (P2 wait time + P3 wait time + P1 wait time)/3
Average waiting time = (0 + 3+ 6) / 3
Average waiting time = 3

Assume all values are in ms

Shortest Job First

- **Operation:** Allocates the CPU to the process that has the **smallest next CPU burst time**.
- **Optimality:** It is provably optimal, giving the minimum average waiting time for a given set of processes.
- **Implementation Challenge:** It is difficult to implement in short-term scheduling because predicting the length of the next CPU burst is impossible.
- **Approximation:** Schedulers typically use **exponential average prediction** based on past burst history to estimate the next burst length.

Shortest Job First

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

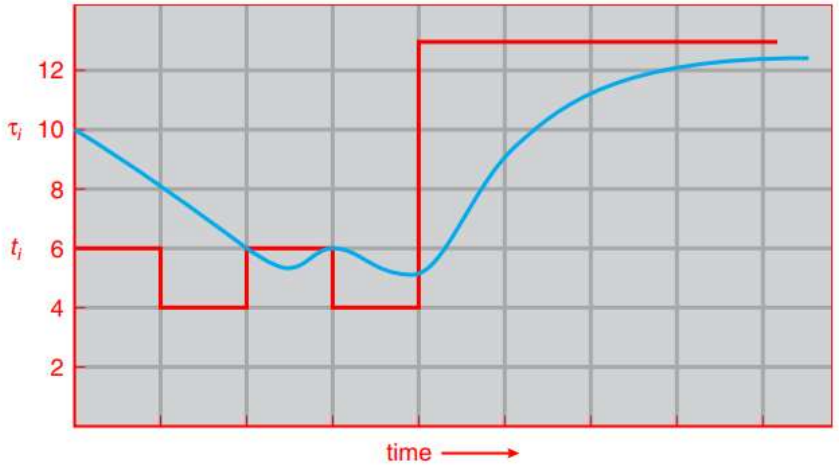


Average waiting time = (P4 wait time + P1 wait time + P3 wait time + P2 wait time)/4
Average waiting time = (0 + 3 + 9 + 16) / 4
Average waiting time = 7

Assume all values are in ms

Exponential Average

- $\tau_{n+1} = \alpha\, t_n + (1-\alpha)\tau_n$.
  - Let t_n be the length of the *n*th CPU burst
  - Let τ_{n+1} be our predicted value for the next CPU burst
  - for α, $0 \le \alpha \le 1$



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Scheduling Algorithms Pre-Emptive

Shortest Remaining Time First or Pre-Emptive Shortest Job First

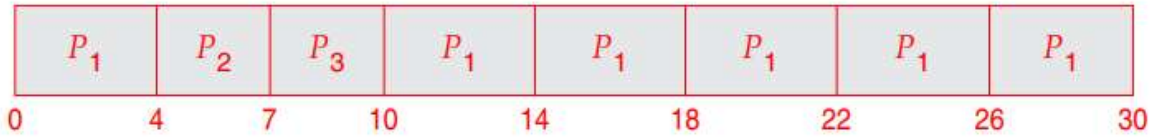
- **Operation:** The preemptive version of SJF. If a new process arrives with a CPU burst shorter than the time remaining on the currently executing process, the current process is preempted.
- **Behavior:** The scheduler always chooses the process with the shortest *remaining* time to completion.
- **Advantage:** Can significantly decrease the average waiting time compared to nonpreemptive SJF, especially for short, incoming jobs.

Round Robin

- **Operation:** Designed for time-sharing systems. Processes are placed in a FIFO ready queue. The CPU scheduler goes around the queue, allocating the CPU to each process for a small time unit called a **time quantum (or time slice)**
- **Preemption:** If the process does not finish within the time quantum, it is preempted and put back at the tail of the ready queue
- **Performance:**
  - Large time quantum - behaves like FCFS
  - Small time quantum - high overhead (too many context switches).
- **Trade-off:** Quantum size must be balanced to minimize overhead while ensuring good response time

Round Robin

Process	Burst Time
P_1	24
P_2	3
P_3	3



Priority Scheduling

- **Operation:** Each process is assigned a **priority** number, and the CPU is allocated to the process with the highest priority (the smallest integer usually represents the highest priority)
- **Preemptive vs. Nonpreemptive:** Can be implemented as both Preemptive priority scheduling will preempt the running process if a higher-priority process arrives
- **Problem: Starvation (Indefinite Blocking):** Low-priority processes may never execute if a steady stream of high-priority processes exists
- **Solution: Aging:** Gradually increases the priority of processes that wait in the system for a long time, preventing starvation

Priority Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2



Priority Scheduling with Round Robin

Process	Burst Time	Priority
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

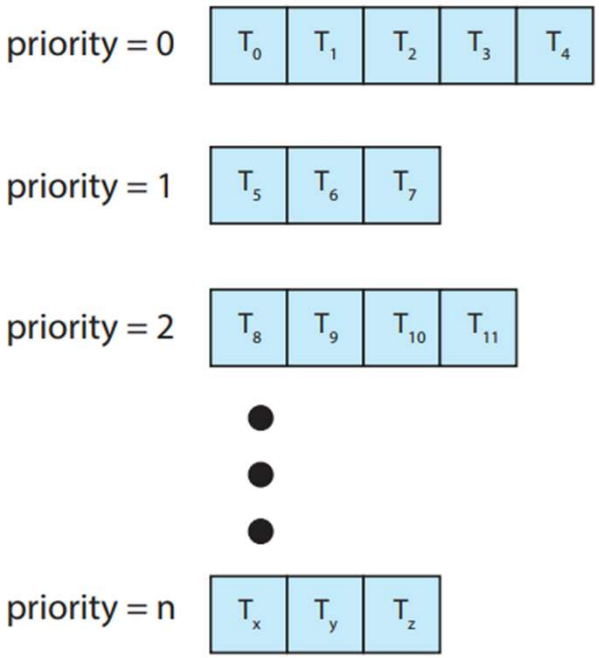


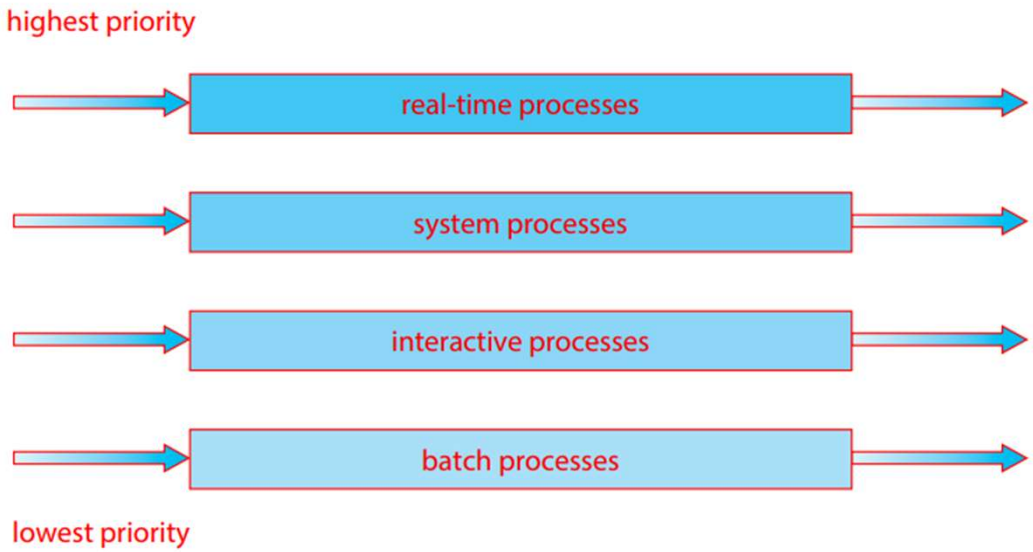
Multilevel Queue and Multilevel Feedback Queue

Multilevel Queue Scheduling

- **Concept:** Divides the ready queue into separate queues based on process characteristics (e.g., foreground/interactive, background/batch).
- **Queue Priority:** Each queue has its own scheduling algorithm (e.g., foreground queue might use RR, background might use FCFS).
- **Scheduling Between Queues:** Fixed-priority preemptive scheduling is often used (e.g., always execute jobs from the foreground queue unless it's empty). This can lead to starvation of lower-priority queues.

Multilevel Queue Scheduling

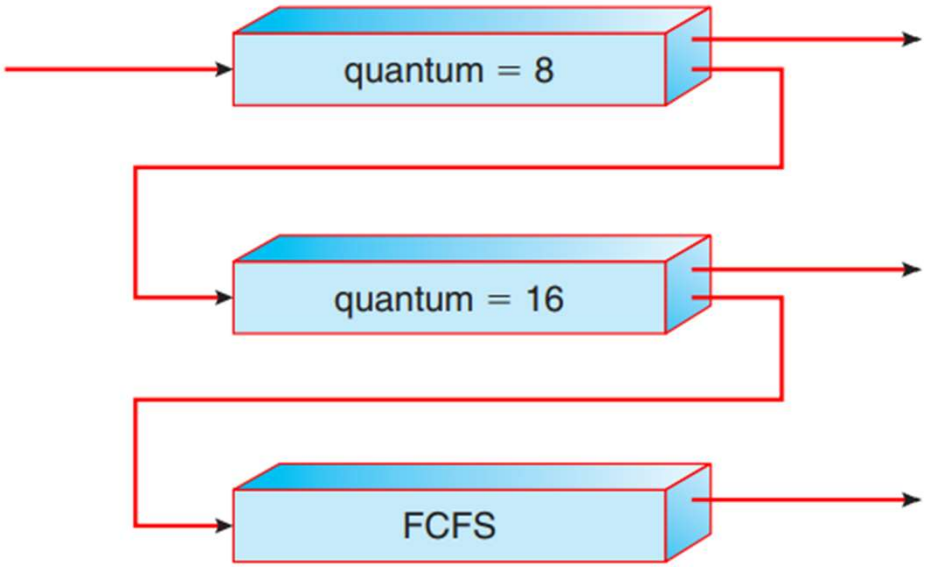




Multilevel Feedback Queue (MLFQ)

- **Concept:** Allows a process to move between different queues based on its behavior. This prevents starvation and separates processes by CPU-burst characteristics.
- **Mechanism:**
  - A process that uses too much CPU time is moved to a lower-priority queue (penalizing CPU-bound processes).
  - A process that waits too long in a low-priority queue can be moved to a higher-priority queue (implementing aging to prevent starvation).
- **Advantage:** The most general and flexible scheduling algorithm. It is configurable by defining parameters like the number of queues, the scheduling algorithm for each queue, and the criteria for moving processes between queues.

Multilevel Feedback Queue Scheduling



Thread Scheduling

- **Difference from Process Scheduling:** Scheduling threads is the same as scheduling processes, but the scope of scheduling must be considered
- **Contention Scope:**
  - **Process-Contention Scope (PCS):** A scheduler decides which **user-level thread** to run on an available **Lightweight Process (LWP)**, which is internal to the process
    - This is typical of Many-to-One and Many-to-Many models
  - **System-Contention Scope (SCS):** The operating system kernel scheduler decides which **kernel-level thread** to run on an available CPU
    - This is typical of One-to-One models and is how modern OS kernels schedule processes.

Multi-Processor Scheduling

- **Homogeneous Processors:** All processors are identical in terms of function and speed. This allows any process in the ready queue to run on any processor.
- **Load Sharing:** The ready queue must be shared among all processors to distribute the workload evenly.
  - **Asymmetric Multiprocessing:** Only one processor (the master server) handles all scheduling decisions, I/O, and other kernel activities.
  - **Symmetric Multiprocessing (SMP):** Each processor is self-scheduling. All processes are in a common ready queue (or each CPU has its own private queue), and each processor selects a process to run.
- **Processor Affinity:** Most SMP systems try to avoid migrating a process from one processor to another because of the cost of invalidating and reloading the cache memory of the new processor.
  - **Soft Affinity:** The OS tries to keep the process on the same processor but doesn't guarantee it.
  - **Hard Affinity:** The process can specify that it should not be moved from its current processor.

Operating System Examples

- **Linux Scheduling:** Uses a scheduling class concept, allowing different algorithms to be implemented. The default scheduler is the **Completely Fair Scheduler (CFS)**, which attempts to approximate the optimal, ideal short-job-first scheduling, emphasizing fairness in resource distribution.
- **Windows Scheduling:** A priority-based, preemptive scheduler. It uses a 32-level priority scheme to determine thread execution.

Algorithm Evaluation

- **Deterministic Modeling:** Takes a specific workload and calculates the performance of each algorithm (e.g., the average waiting time). Simple, but limited because the workload can change.
- **Queuing Models:** Uses mathematical formulas (e.g., Little's formula) to calculate performance characteristics like CPU utilization, average waiting time, and throughput under specific probability distributions of processes and CPU bursts.
- **Simulations:** Involves programming a model of the computer system and running it with real or randomly generated data to evaluate performance. More accurate but expensive.
- **Implementation:** The most accurate method. Involves coding the scheduling algorithm and placing it in a real operating system, then measuring real-world performance.

Summary

- CPU scheduling manages the CPU to switch processes efficiently, maximizing throughput and minimizing latency.
- Performance is measured by **CPU Utilization, Throughput, Turnaround Time, Waiting Time, and Response Time**.
- **Major Algorithms:**
  - **FCFS:** Simple, but suffers from the **Convoy Effect**.
  - **SJF/SRTF:** Mathematically **optimal** (minimum average waiting time), but requires predicting the next burst length.
  - **Priority:** Effective, but must implement **Aging** to prevent starvation.
  - **Round-Robin (RR):** Essential for time-sharing systems, uses a **time quantum** to ensure fairness and responsiveness.
- **Advanced Scheduling: Multilevel Feedback Queues (MLFQ)** are used in modern OSes to adapt to process behavior and prevent starvation.
- **Multiprocessor Systems:** Focus on **Load Sharing** and **Processor Affinity** to balance workload while minimizing cache invalidation overhead.